

.Scora

Code Luck

Build Your Lucky Future

Technical Architecture & Next.js Plan

MVP stack, dependency topology, security boundaries & delivery sequence

1. Technical Direction

Scora will be implemented as a modern Next.js web application. Next.js is the primary application framework for the MVP, providing the routing, server-rendered pages, API boundaries, authentication integration points, and deployment path needed for the product experience.

Technical decision: use Next.js as the overall web application framework. Keep the architecture modular so assessment execution, code evaluation, trust signals, and search can evolve independently.

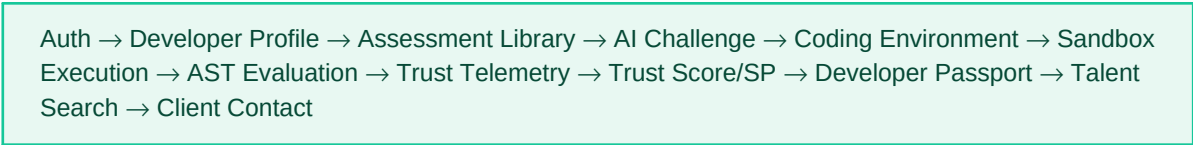
2. Recommended MVP Stack

Layer	MVP choice	Responsibility
Web application	Next.js (App Router)	Pages, layouts, routing, server/client boundaries.
Language	TypeScript	Typed domain models, safer API contracts, maintainable feature modules.
UI system	Existing Scora design system	RTL layout, tokens, reusable components, responsive states.
API boundary	Next.js Route Handlers / service modules	Auth, assessments, submissions, passport, search, contact.
Execution service	Isolated sandbox service	Run code and tests away from the web process.
Evaluation	AST analysis worker	Complexity, duplication, maintainability, security signals.
Data layer	Relational database (final provider TBD)	Users, assessments, attempts, scores, passports, contacts.
Deployment	Node-compatible hosting	Web app, workers, environment variables, observability.

3. High-Level Architecture

Boundary	Modules
Experience	Landing, auth, onboarding, developer dashboard, client discovery, passport, contact.
Application	Profiles, assessment orchestration, submissions, score calculation, search, notifications.
Evaluation	Challenge generator, test runner, AST analyzer, telemetry collector, trust engine.
Infrastructure	Database, isolated containers, queues/workers, logs, rate limits, monitoring.

4. Dependency Flow



5. Repository Structure

Folder	Purpose
app/ or src/app/	Next.js routes, layouts, loading/error states, route handlers.
components/	Shared UI and Scora-specific components.
features/	Domain modules: auth, assessment, passport, discovery, contact.
lib/	Database, auth, API clients, validation, telemetry helpers.
workers/	Sandbox and AST evaluation jobs kept outside request rendering.
public/ and assets/	Static assets, icons, illustrations, exported design references.
docs/ and deliverables/	Product, research, technical, design, and Phase 2 submission documents.

6. Assessment Runtime

- Next.js creates the assessment session and returns challenge metadata.
- The browser coding environment loads the repository, language mode, timer, and visible tests.
- Submission is sent to an isolated sandbox; the web process never executes untrusted code directly.
- Test output, syntax anomalies, and execution traces are stored against the attempt.
- AST and trust workers compute maintainability and behavior signals asynchronously.
- The result service updates Skill Points, Trust Score, and Passport state after validation.

7. Trust & Security Boundaries

- Use role-based access for Developer, Client, Company, Reviewer, and Admin areas.
- Isolate code execution with network restrictions, CPU/memory/time limits, and disposable containers.
- Treat tab switches, paste entropy, and cadence as supporting signals; never expose private telemetry details unnecessarily.
- Log score changes and verification events for an auditable Passport history.
- Keep secrets in environment variables; commit only an .env.example file.

Fail-safe: if the AST parser cannot process a submission, fall back to test execution scoring, flag the syntax anomaly, and keep the attempt reviewable.

8. Delivery Sequence

Step	Implementation focus
1	Next.js shell, RTL design system, auth, roles, profile onboarding.
2	Assessment library, challenge session, timer, save state, submission API.
3	Sandbox runner, test output, AST worker, score calculation.
4	Passport, developer discovery, filters, contact flow, responsive QA.
5	Observability, security review, performance checks, production deployment.

9. Technical Decisions Still Open

Finalize the database provider, authentication provider, queue/worker runtime, sandbox technology, AST parser packages, deployment target, and observability stack during implementation kickoff. These decisions should be recorded in the repository README when confirmed.